

■ DSA Mental Framework

Weeks 1–6 Complete Reference · Python · 80+ Problems

This framework contains everything you need to solve DSA problems at your fingertips. Read it once, drill it daily, own it forever.

LAYER 1 — THE ANALOGY BANK

Lock these in. When you see a problem — the analogy fires instantly.

TOOL	ANALOGY	USE WHEN
SET	Bouncer's stamp log "Have I seen this face?"	YES/NO existence check $O(1)$
DICT	Phone book "Give me Ravi's number"	Key $\rightarrow$ Value lookup $O(1)$
COUNTER	Tally chart "How many times?"	Frequency counting
DEFAULTDICT	Phone book that auto-creates pages	Group items under a key
LIST	Numbered lockers "Give me locker #3"	Indexed storage $O(1)$
STACK	Plates on a table Last in = First out (LIFO)	Brackets, undo, state saving
QUEUE	Ticket counter line First in = First served (FIFO)	BFS, level order traversal
MONO STACK	Waiting room Pop when answer found	Next greater/smaller element
HEAP	Hospital emergency room Most critical patient first	K largest/smallest, median
LINKED LIST	Treasure hunt Each clue points to next	Dynamic insertions $O(1)$ head
TREE	Family tree Parent has two children	Hierarchical data, recursion
BST	Dictionary book Left=smaller, Right=larger	$O(\log n)$ search
BINARY SEARCH	Dictionary lookup Open middle, go left or right	Sorted array search $O(\log n)$
TWO POINTERS	Two fingers on array Move based on condition	Pairs, palindromes, sorted
SLIDING WINDOW	Cardboard frame Move across frozen array	Best subarray problems
PREFIX SUM	Car odometer Accumulates, never resets	Range sum queries $O(1)$

MERGE SORT	Divide army then conquer Split → sort → merge	Guaranteed $O(n \log n)$
INTERVALS	Timeline of meetings Sort by start → process	Overlap detection, merging

LAYER 2 — THE DECISION ENGINE

See problem → fire answer instantly. Ask these 2 questions first.

WHAT AM I STORING?

QUESTION	TOOL
Just YES/NO existence?	SET
Key maps to value?	DICT
Count occurrences?	COUNTER
Group under shared key?	DEFAULTDICT
Position matters?	LIST
Remember + reverse order?	STACK
Process in arrival order?	QUEUE (deque)
Min/max repeatedly?	HEAP

HOW AM I MOVING?

SITUATION	PATTERN
Sorted array + find pair?	TWO POINTERS (opposite ends)
Move elements in place?	TWO POINTERS (slow/fast)
Best subarray size k given?	FIXED SLIDING WINDOW
Longest/shortest condition?	VARIABLE SLIDING WINDOW
Range sum queries?	PREFIX SUM
Subarray sum = k?	PREFIX SUM + DICT
Next greater/smaller?	MONOTONIC STACK
Level by level tree?	BFS + QUEUE
Depth first tree?	DFS + RECURSION
Sorted array find target?	BINARY SEARCH
K largest/smallest?	HEAP size K
Overlapping intervals?	SORT BY START + MERGE
Cycle in linked list?	SLOW/FAST POINTERS
Middle of linked list?	SLOW/FAST POINTERS

LAYER 3 — THE ONE-LINE RULES

These rules solve problems instantly. Memorise every single one.

WEEK 1 — TOOLS

- dict/set lookup = $O(1)$ always
- list search = $O(n)$ always
- Two separate loops = $O(n)$ not $O(n^2)$
- Trade space for time = use dict/set
- Counter('aab') = {'a':2, 'b':1} instantly

WEEK 2 — PATTERNS

- $R - L + 1$ = window size (both endpoints included)
- Fixed window = 'size k' in problem
- Variable window = 'longest/shortest'
- Shrink when MET = minimise problems
- Shrink when BROKEN = maximise problems
- $\text{prefix}[R+1] - \text{prefix}[L]$ = range sum $O(1)$

WEEK 3 — STRUCTURES

- Stack top = `stack[-1]` in Python
- Monotonic stack = $O(n)$ despite while loop (each element in+out once)
- BFS uses queue = FIFO natural for levels
- `len(queue)` at loop start = exact nodes this level
- Decreasing stack → finds next GREATER element
- Increasing stack → finds next SMALLER element

WEEK 4 — LINKED LISTS

- Dummy node → return `dummy.next` always
- Three pointers = SAVE next → REVERSE → MOVE both
- Slow/fast middle = fast 2x speed → slow at middle
- Recursion cost = $O(n)$ hidden call stack space
- Floyd phase 2 = reset slow to head → move both 1 step
- Iterative = $O(1)$ space, Recursive = $O(n)$ space

WEEK 5 — TREES

- Tree template: if not node → return base; left=solve(left); right=solve(right); return combine
- BST inorder = sorted order ALWAYS
- Validate BST = pass min/max boundaries, NOT just check children
- $\text{left} \leq \text{right}$ in binary search = handles single element case
- Rotated array = ONE half always sorted → eliminate wrong half
- BFS for shortest path, DFS for depth problems

WEEK 6 — SORTING + HEAPS

- Merge sort = $O(n \log n)$ ALWAYS (guaranteed)
- Quick sort = $O(n^2)$ worst case (avoid for interviews)
- K largest = MIN heap size K (pop smallest to keep largest)

- K smallest = MAX heap size K
- heap[0] = Kth largest ALWAYS
- Median stream = two heaps (small=lower, large=upper) balanced
- Intervals = SORT BY START TIME FIRST (no exceptions!)
- Non-overlapping intervals = sort by END time instead
- Meeting rooms = heap stores END TIMES, heap size = rooms needed

LAYER 4 — THE 5 STEP PROCESS

Never skip. Ever. This is the difference between solving and struggling.

STEP	ACTION	DETAIL
1	Read problem TWICE	Understand before anything else
2	Draw example on PAPER	Small input, trace by hand
3	Answer 3 Questions	What to STORE? How to MOVE? Time+Space?
4	TRACE on paper	Walk through your approach manually
5	THEN open editor	Code matches your trace exactly
!	If stuck → Step 2	Never Google before 20 min attempt

LAYER 5 — PROBLEM PATTERNS BY FEEL

When you read a problem — it FEELS like one of these. Train this instinct.

FEEL / KEYWORD IN PROBLEM	APPROACH
'find if something exists'	SET membership check O(1)
'two numbers sum to X'	DICT complement: seen[target-num]
'best/longest/shortest subarray'	SLIDING WINDOW (variable if condition, fixed if k given)
'sum between positions L and R'	PREFIX SUM: prefix[R+1] - prefix[L]
'next greater/smaller element'	MONOTONIC STACK (decreasing→greater, increasing→smaller)
'process tree level by level'	BFS + freeze len(queue) each level
'find path root to leaf'	DFS recursion with running total
'find in sorted array'	BINARY SEARCH: left <= right, eliminate half
'K largest or K most frequent'	MIN HEAP size K: heap[0] = Kth largest
'cycle or middle of linked list'	SLOW/FAST POINTERS (fast 2x speed)
'merge or count overlapping intervals'	SORT BY START → check next.start <= prev.end
'median of stream'	TWO HEAPS: small=max heap lower, large=min heap upper
'minimum rooms/containers needed'	HEAP stores end times, heap size = answer
'passengers at each stop'	DIFFERENCE ARRAY: +at start, -at end, prefix sum

LAYER 6 — NEVER DO THESE AGAIN

WRONG ■	RIGHT ■	WHY
return inside loop	build inside → return OUTSIDE	exits after first iteration
list.pop(0) for queue	deque.popleft()	pop(0) is O(n), popleft is O(1)
s.join(list)	"".join(list)	s adds input string between each char
store VALUES in mono stack	store INDICES	need position AND value for width calc
while left < right	while left <= right	misses single element where left==right
check only direct children in BST	pass min/max boundaries down	subtree can violate constraint
left > node in BST	left < node < right	BST rule: smaller left, larger right
Google when stuck	paper → think 20 min → ask	Googling = zero learning
pick tool before understanding	understand → 3 questions → tool	tool follows from problem
code before tracing	trace on paper → then code	paper catches bugs before they happen
k inside add method (KthLargest)	self.k always	k not in scope outside __init__

LAYER 7 — COMPLEXITY CHEAT SHEET

OPERATION	TIME	SPACE
Hash map get/set	O(1)	O(n)
Set membership	O(1)	O(n)
List index access	O(1)	O(1)
List search	O(n)	O(1)
Stack push/pop	O(1)	O(n)
Heap push/pop	O(log n)	O(n)
Heap get min/max	O(1)	O(n)
Binary search	O(log n)	O(1)
Merge sort	O(n log n)	O(n)
Quick sort avg	O(n log n)	O(log n)
BFS / DFS	O(n)	O(n)
Prefix sum build	O(n)	O(n)
Prefix range query	O(1)	O(1)
Monotonic stack	O(n)	O(n)
Sliding window	O(n)	O(1) or O(k)
Two pointers	O(n)	O(1)
Sort intervals	O(n log n)	O(1)
Single loop	O(n)	—
Two nested loops	O(n ²)	—
Two separate loops	O(n)	—

LAYER 8 — ALL 80+ PROBLEMS GROUPED BY PATTERN

See the pattern → remember the problem. See the problem → remember the pattern.

SET PROBLEMS

Contains Duplicate · Jewels and Stones · Intersection of Two Arrays · Longest Substring No Repeat

DICT PROBLEMS

Two Sum · Word Pattern · First Unique Character · Subarray Sum = K (+ prefix sum)

COUNTER + DEFAULTDICT

Ransom Note · Top K Frequent Elements · Valid Anagram · Group Anagrams · Four Sum II · Majority Element

TWO POINTER PROBLEMS

Two Sum II (sorted) · Valid Palindrome · Valid Palindrome II · 3Sum · Move Zeroes · Remove Duplicates · Sort Array By Parity

SLIDING WINDOW PROBLEMS

Max Average Subarray (fixed) · Longest Substring No Repeat (variable) · Longest Repeating Char Replace (variable) · Max Vowels in Substring (fixed) · Minimum Size Subarray Sum (variable) · Find All Anagrams (fixed + Counter)

PREFIX SUM PROBLEMS

Range Sum Query · Subarray Sum = K · Product Except Self · Car Pooling (difference array) · Continuous Subarray Sum

STACK PROBLEMS

Valid Parentheses · Min Stack · Decode String · Asteroid Collision · Make The String Great

MONOTONIC STACK

Daily Temperatures · Next Greater Element I · Largest Rectangle in Histogram · Sliding Window Maximum

BFS PROBLEMS

Level Order Traversal · Right Side View · Minimum Depth · Meeting Rooms II

DFS PROBLEMS

Max Depth · Invert Binary Tree · Same Tree · Path Sum · Balanced Binary Tree · Max Path Sum

BST PROBLEMS

Validate BST · Lowest Common Ancestor · Kth Smallest in BST

BINARY SEARCH

Binary Search · Search Rotated Array · Find Minimum Rotated

LINKED LIST PROBLEMS

Reverse Linked List · Merge Two Sorted Lists · Linked List Cycle · Middle of Linked List · Remove Nth From End · Palindrome Linked List · Reorder List · Add Two Numbers · Copy Random List · LRU Cache · Linked List Cycle II · Flatten Multilevel List

HEAP PROBLEMS

Kth Largest Element · K Closest Points · Task Scheduler · Find Median From Stream · Kth Largest in Stream · IPO

SORTING + INTERVALS

Sort an Array (Merge Sort) · Sort Colors · Merge Intervals · Non-overlapping Intervals · Insert Interval · Meeting Rooms · Car Pooling

LAYER 9 — THE DAILY 5-MINUTE DRILL

Every morning. No exceptions. 2 weeks → automatic forever.

MORNING ROUTINE	
Step 1	Open blank paper — no notes, no phone
Step 2	Write from memory: 8 tools + when to use each
Step 3	Write from memory: 10 patterns + when to use each

Step 4	Write from memory: 5 step process
Step 5	Write from memory: 5 biggest mistakes to avoid
Step 6	Check against this framework
Step 7	Whatever you forgot → rewrite it 3 times
RESULT	After 2 weeks → fires automatically in interviews